



UNIVERSIDAD PRIVADA DE TACNA

**FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas**

**INFORME DE LABORATORIO
“EJERCICIO CON LINQ”**

Curso: Sistemas de Información

Docente: Ing. Breno Luque Zuñiga

Peralta Sebán, Daniel Sebastián (2024016803)

**Tacna – Perú
2026**



INDICE

I. INFORMACIÓN GENERAL	3
- Objetivos	3
- Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
Programa: Imprimir un elemento:.....	4
1. Diseño del Programa.....	4
2. Tabla de Controles.....	4
3. Código del Programa.....	5
IV. ANALISIS E INTERPRETACION DE RESULTADOS	8
CONCLUSIONES	8
RECOMENDACIONES	8
WEBGRAFIA	8



INFORME DE LABORATORIO

TEMA: EJERCICIO CON LINQ

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - Creación de una aplicación capaz de gestionar y dar de alta personas.
 - Conocer el uso de listas (List) como estructura de datos dinámicas.
 - Realizar consultas mediante LINQ para filtrar y analizar información.
 - Validar el dato escrito por el usuario.
 - Utilizar controles gráficos como DataGridView para mostrar información.
 - Calcular valores estadísticos como la media.
- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con sistema operativo Windows 8
 - Visual Studio
 - Lenguaje de programación C#.
 - .NET Framework

II. MARCO TEORICO

List: Es una colección dinámica que permite almacenar elementos de un mismo tipo, con capacidad de crecer según sea necesario.

LINQ (Language Integrated Query): LINQ permite realizar consultas sobre colecciones de datos de forma sencilla y eficiente utilizando expresiones lambda.

Expresiones Lambda: Son funciones anónimas que permiten escribir código más compacto y legible.

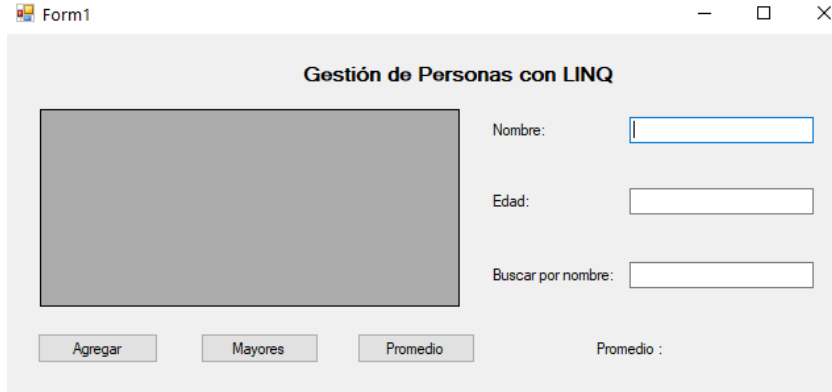
DataGridView: Permite visualizar datos en forma tabular, facilitando la interacción con el usuario.

Validación de datos: Es fundamental para garantizar que la información ingresada sea correcta antes de procesarla.

III. PROCEDIMIENTO

Programa: Buscador de letras:

1. Diseño del Programa



2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	Gestión de Personas con LINQ
	Font	bold type
Label2	Text	Nombre:
Label3	Text	Edad:
Label4	Text	Buscar por nombre:
Label5	Name	lblResultado
	Text	Promedio :
Button1	Name	btnAgregar
	Text	Agregar
Button2	Name	btnMayores
	Text	Mayores
Button3	Name	btnPromedio
	Text	Promedio
Textbox1	Name	txtNombre
Textbox2	Name	txtEdad
Textbox3	Name	txtBuscar
dataGridView1	Name	dataGridView1

3. Código del Programa

Funcionalidad 1: Definición de la clase Persona

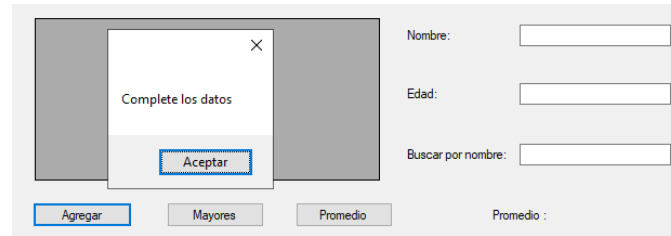
Se aplica programación orientada a objetos para representar cada registro.

```
4 referencias
public class Persona
{
    2 referencias
    public string Nombre { get; set; }
    3 referencias
    public int Edad { get; set; }
}
```

Funcionalidad 2: Agregar Persona

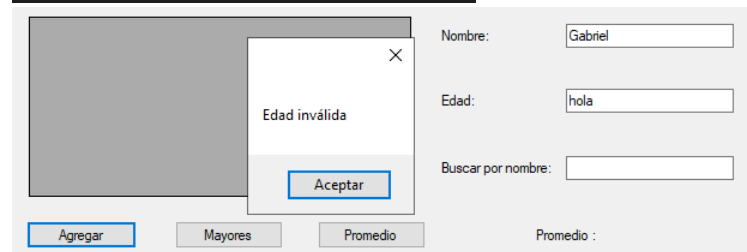
Se verifica que los datos estén completos.

```
private void btnAgregar_Click(object sender, EventArgs e)
{
    if (txtNombre.Text == "" || txtEdad.Text == "")
    {
        MessageBox.Show("Complete los datos");
        return;
    }
}
```



Se asegura que la edad sea un número válido.

```
int edad;
if (!int.TryParse(txtEdad.Text, out edad))
{
    MessageBox.Show("Edad inválida");
    return;
}
```



Se instancia un nuevo objeto Persona.

```
Persona p = new Persona()
{
    Nombre = txtNombre.Text,
    Edad = edad
};
```



Se agrega a la lista Personas, se actualiza el DataGridView y se garantiza que los datos sean válidos antes de almacenarlos.

```
ListaPersonas.Add(p);

dataGridView1.DataSource = null;
dataGridView1.DataSource = ListaPersonas;

txtNombre.Clear();
txtEdad.Clear();
}
```

	Nombre	Edad
▶	Gabriel	23
	Juan	12
	Anna	24

Nombre
Edad:
Buscar

Agregar Mayores Promedio

Funcionalidad 3: Filtrar Mayores de Edad

Se utiliza LINQ para obtener solo las personas mayores de edad.

```
1 referencia
private void btnMayores_Click(object sender, EventArgs e)
{
    var mayores = listaPersonas
        .Where(p => p.Edad >= 18)
        .ToList();

    dataGridView1.DataSource = null;
    dataGridView1.DataSource = mayores;
}
```

	Nombre	Edad
▶	Gabriel	23
	Anna	24

No
Ed
Bus

Agregar Mayores Promedio



Funcionalidad 4: Búsqueda en tiempo real

Permite filtrar la lista dinámicamente según el texto ingresado.

```
1 referencia
private void txtBuscar_TextChanged(object sender, EventArgs e)
{
    var filtro = listaPersonas
        .Where(p => p.Nombre.ToLower()
            .Contains(txtBuscar.Text.ToLower()))
        .ToList();

    dataGridView1.DataSource = null;
    dataGridView1.DataSource = filtro;
}
```

	Nombre	Edad
▶	Anna	24

Nombre:

Edad:

Buscar por nombre:

Funcionalidad 5: Cálculo de Promedio

Revisa que la lista este primero llena sino mostrara un texto

```
1 referencia
private void btnPromedio_Click(object sender, EventArgs e)
{
    if (listaPersonas.Count == 0)
    {
        lblResultado.Text = "Sin datos";
        return;
    }
}
```

Nombre:

Edad:

Buscar por nombre:

AgregarMayoresPromedioSin datos

Se utiliza LINQ para calcular un valor estadístico de forma sencilla.

```
double promedio = listaPersonas.Average(p => p.Edad);
lblResultado.Text = "Promedio : " + promedio.ToString("0.0");
}
```

	Nombre	Edad
▶	Gabriel	23
	Juan	12
	Anna	24

Nombre:

Edad:

Buscar por nombre:

AgregarMayoresPromedioPromedio : 19.7



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

- El sistema permite registrar personas de un modo correcto.
- Los datos se muestran en tiempo real en el DataGridView.
- Las consultas LINQ permiten filtrar y analizar información de una forma muy eficaz.

¿Que se ha encontrado?

- LINQ ayuda a manejar más fácilmente los datos.
- La validación evita problemas en la aplicación.
- La búsqueda dinámica ayuda a mejorar la experiencia del usuario.
- El cálculo de promedio permite acceder a la información más rápidamente.

CONCLUSIONES

- Se logra tener un sistema de gestión de personas sencillo y funcional, mediante listas dinámicas.
- LINQ se comporta como una potente herramienta para realizar consultas a colecciones.
- El uso de DataGridView permite ver y manipular datos con facilidad.
- Las expresiones lambda permiten obtener una escritura más limpia y eficaz.
- El sistema es extensible, por lo que permite añadir funcionalidad a partir del mismo.
- La combinación de lógica-interfaz-consultas da muestra de un nivel avanzado de programación.
- Este tipo de aplicaciones es base para sistemas más complejos como registros administrativos o bases de datos.

RECOMENDACIONES

- Implementar almacenamiento en base de datos.
- Agregar opciones de edición y eliminación.
- Mejorar la interfaz gráfica.

WEBGRAFIA

BillWagner. (s. f.). Operadores y expresiones: enumerar todos los operadores y expresiones - C# reference. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>



Arreglos en C#. (2014). C# Paso a Paso. Recuperado de <https://csharp-facilito.blogspot.com/2013/07/arreglos-en-c-sharp.html>

Turrado, J. (2022). Introducción rápida a LINQ con C#: manejar información en memoria nunca fue tan sencillo - campusMVP.es. campusMVP.es. Recuperado de https://www.campusmvp.es/recursos/post/introduccion-rapida-a-linq-con-c-sharp.aspx?srsId=AfmBOogx5_WZg2mtHdV0_YFJiBZsZ5dj_t3Us87MablaeW1k-0upk2xK